```
Hex:
0x48 0x45 0x4c 0x4c

Integer:
1280066888 // on a little-endian CPU e.g. Intel
1212501068 // on a big-endian CPU
```

We can see that the same data – the same byte sequence – can represent different information depending on the context in which it is evaluated. Programming languages will generally store metadata (data about the data) about their data values. One of the most fundamentally important pieces of metadata is something called *data type*.

Data types are attributes that an interpreter (in our case, the PHP binary) or compiler uses to determine which operations can be safely performed on that data; "safe" in this case meaning "do what is expected." Numerical types can have mathematical operations performed on them, strings can be combined and searched, booleans are used in logical expressions, and so on.

The corollary to this is that non-compatible types and operations require that either the data type is changed to something compatible or the operation is prevented (usually causing termination of the program; in PHP's case, a "Fatal error" message is often the resulting output). How this comes about might depend on the language or the way the language is configured to behave when the program compiles or runs. There are usually also limitations on which type conversions are possible.

PHP 8 Data Types

Some languages, like C, are *strongly typed* (also known as *strictly* or *statically* typed). When variables are declared in C, they must always have a type declaration, and operations that are incompatible with the data type will cause errors. If you wish to treat a sequence of characters as a number, in C, it first has to be converted (or *cast*) to the required type.

PHP, however, is a *dynamically typed* (also known as *loosely* or *weakly* typed) language. It isn't necessary to declare type when creating a PHP variable; in fact, the language doesn't even have the syntax for doing this. PHP will automatically determine the data type for you from the values supplied when creating the variable, as we shall see later in the chapter.